

# LP-Spec: Leveraging LPDDR PIM for Efficient LLM Mobile Speculative Inference with Architecture-Dataflow Co-Optimization

Siyuan He\*, Zhantong Zhu\*, Yandong He, Tianyu Jia<sup>†</sup>  
 School of Integrated Circuits  
 Peking University  
 Beijing, China

<sup>†</sup>Corresponding Email: tianyu.jia@pku.edu.cn

**Abstract**—LLM inference on mobile devices faces extraneous challenges due to limited memory bandwidth and computational resources. To address these issues, speculative inference and processing-in-memory (PIM) techniques have been explored at the algorithmic and hardware levels. However, speculative inference results in more compute-intensive GEMM operations, creating new design trade-offs for existing GEMV-accelerated PIM architectures. Furthermore, there exists a significant amount of redundant draft tokens in tree-based speculative inference, necessitating efficient token management schemes to minimize energy consumption. In this work, we present *LP-Spec*, an architecture-dataflow co-design leveraging hybrid LPDDR5 performance-enhanced PIM architecture with draft token pruning and dynamic workload scheduling to accelerate LLM speculative inference. A near-data memory controller is proposed to enable data reallocation between DRAM and PIM banks. Furthermore, a data allocation unit based on the hardware-aware draft token pruner is developed to minimize energy consumption and fully exploit parallel execution opportunities. Compared to end-to-end LLM inference on other mobile solutions such as mobile NPUs or GEMV-accelerated PIMs, our *LP-Spec* achieves 13.21 $\times$ , 7.56 $\times$ , and 99.87 $\times$  improvements in performance, energy efficiency, and energy-delay-product (EDP). Compared with prior AttAcc PIM and RTX 3090 GPU, *LP-Spec* can obtain 12.83 $\times$  and 415.31 $\times$  EDP reduction benefits.

**Index Terms**—Large Language Model, Speculative Inference, Processing-in-Memory, Inference Acceleration

## I. INTRODUCTION

Large language models (LLMs) have rapidly advanced across multiple application domains and exhibited remarkable performance [1]–[3]. Recently, LLMs have transitioned from large-scale cloud infrastructures to consumer-level mobile devices, such as smartphones or laptops [4], [5]. Mobile devices feature dedicated AI acceleration hardware such as neural processing units (NPUs) to handle computation demands of AI tasks, including LLM inference. However, mobile hardware must adhere to stringent energy, area, and inference latency constraints. With limited memory bandwidth, memory and computation capacity, efficient LLM deployment on mobile devices necessitates cross-stack optimization approaches.

To enable efficient LLM inference on mobile devices, significant algorithm-level advances have emerged. Beyond

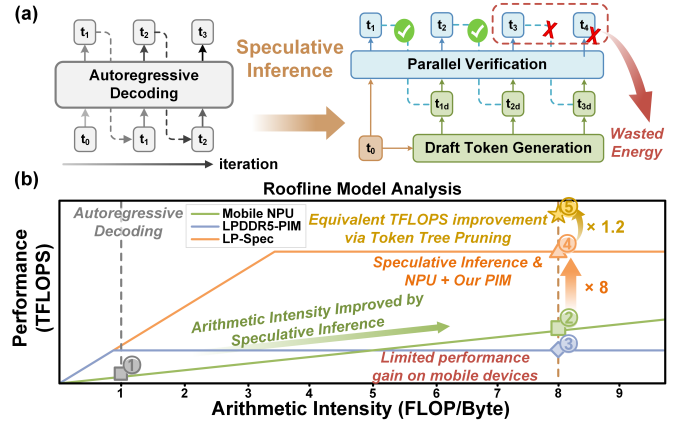


Fig. 1. (a) Autoregressive decoding and speculative inference, (b) challenges of speculative inference on existing mobile devices and our design target.

established techniques such as quantization [6]–[8] and weight pruning [9]–[11], speculative inference [12]–[17] has recently emerged as a promising approach to overcome the sequential bottleneck of autoregressive decoding. As shown in Fig. 1(a), conventional autoregressive decoding generates one token at a time, with each step depending on the previous output. Speculative inference, by contrast, predicts multiple future tokens and verifies them in parallel, achieving around 2-3 $\times$  speedup over vanilla autoregressive decoding.

Fig. 1(b) utilize a roofline model to illustrate LLM inference deployment on mobile devices. Conventional autoregressive decoding is the primary contributor to LLM inference latency [18] due to limited memory bandwidth (Fig. 1(b) ①). Speculative inference addresses this by transforming general matrix-vector multiplication (GEMV) operations into more compute-intensive general matrix-matrix multiplication (GEMM) operations (Fig. 1(b) ②). However, resource and power-constrained mobile devices typically only incorporate LPDDR5 DRAM modules with limited bandwidth, e.g., 51.2 GB/s, and rely on mobile NPUs for LLM inference, resulting in sub-optimal performance gains (②). Furthermore, increasing the number of draft tokens, i.e., speculation length, also raises the computa-

\*Equal contribution to this work.

tional burden for verification, and many tokens are ultimately rejected, leading to wasted energy.

To address this memory bandwidth bottleneck, recent advances in DRAM-based processing-in-memory (PIM) technology have been introduced, leveraging high internal bandwidth by enabling computation directly inside DRAM dies. However, current PIM designs face two key limitations for enabling efficient LLM speculative inference on mobile devices. *First, prevalent PIM architectures are predominantly optimized for GEMV operations*, as seen in SK Hynix’s GDDR6-PIM [19] and Samsung’s HBM2-PIM [20]. This GEMV-centric architecture is inefficient for speculative inference, which involves a higher proportion of GEMM operations (Ⓔ). *Second, current PIM architectures prevent NPU and PIM parallel execution*, as DRAM memory access is blocked by PIM operations, thereby limiting overall hardware utilization. *Consequently, novel PIM architectural innovations and dynamic scheduling schemes are desired to support efficient deployment of speculative inference on mobile devices.*

In this work, we introduce *LP-Spec*, an architecture-dataflow co-design leveraging hybrid LPDDR5-based PIM architecture with dynamic workload scheduling and runtime data reallocation to accelerate LLM speculative inference. First, we present a hybrid LPDDR5-PIM module with DRAM and PIM ranks and near-data memory controller. Then, we propose a hardware-aware draft token pruning scheme to enhance computational efficiency and inference latency. Given varying speculation lengths at runtime, we introduce a dynamic workload scheduler with data reallocation to fully exploit NPU-PIM parallel execution and maximize hardware utilization. We evaluate *LP-Spec* against prior NPU and NPU-PIM systems [20]–[23], showing up to  $13.21\times$  performance improvement and  $7.56\times$  better energy efficiency. Compared with prior PIM-based design AttAcc [24] and Nvidia RTX 3090 GPU, *LP-Spec* achieves  $12.83\times$  and  $415.31\times$  EDP reduction.

The contributions of our work are summarized as follows:

- We present *LP-Spec*, an NPU-PIM heterogeneous architecture with GEMM-enhanced LPDDR5-PIM to improve computational performance for speculative inference.
- We design a near-data memory controller to support simultaneous PIM computation and DRAM access, enabling data reallocation at runtime.
- We present a hardware-aware runtime draft token pruning scheme to minimize redundant computation and energy consumption cost.
- We evaluate our design against mobile NPU and PIM baselines, showing up to  $99.87\times$  EDP gain. Our *LP-Spec* also shows  $12.83\times$  and  $415.31\times$  better EDP than prior AttAcc PIM and RTX 3090 GPU.

## II. BACKGROUND

### A. Processing-in-Memory (PIM)

Conventional architectures like GPUs face performance bottlenecks for memory-intensive operations due to restricted memory bandwidth. PIM designs address this challenge by

TABLE I  
COMPARISON BETWEEN DIFFERENT TYPES OF SPECULATIVE INFERENCE.

| Methods        | Drafting Scheme | Token Structure |
|----------------|-----------------|-----------------|
| SpecDec [17]   | DLM             | Sequence-based  |
| Blockwise [30] | Decode Heads    | Sequence-based  |
| SpecInfer [12] | DLM             | Tree-based      |
| Medusa [16]    | Decode Heads    | Tree-based      |

integrating processing units within or near DRAM banks [19], [20], [25], [26]. By leveraging high internal bandwidth of DRAM banks and reducing off-chip data movement, PIM significantly improves computation throughput and energy efficiency. For instance, while a single  $\times 64$  LPDDR5 DRAM chip with  $4 \times 16$  LPDDR5 DRAM dies provides 51.2 GB/s of external I/O bandwidth, its internal all-bank bandwidth can reach up to 409.6 GB/s [27]. Moreover, data transfers within DRAM consume only 15% of the energy required for off-DRAM transfers [23]. Recent PIM-based system architectures have explored combining PIM with xPUs, e.g., GPUs and NPUs, to optimize LLM inference [24], [28], [29]. For example, AttAcc [24] integrates GEMV and Softmax units within HBM stacks to accelerate attention layers, employing optimized pipelining and parallelism strategies to improve autoregressive decoding. Similarly, SpecPIM [29] explores the architectural and workload mapping design space to enhance draft language model-based (DLM) speculative inference.

Prior work focused on leveraging GPU-PIM heterogeneous systems to accelerate large-scale LLM inference for cloud serving scenarios, in which the compute-intensive FC layers are executed in GPUs and the memory-intensive attention layers are offloaded to PIM. However, these approaches are less effective when adapted to mobile devices, due to the much lower memory bandwidth and limited computational resources. Mobile NPUs are often limited by off-chip memory bandwidth while conventional GEMV-targeted PIMs struggle with arithmetic-intensive GEMM operations. This limitation motivates us to explore architecture-dataflow co-optimization for speculative LLM inference on resource-constrained mobile devices.

### B. Speculative Inference

LLM inference typically employs an autoregressive decoding scheme, generating one output token per iteration due to the data dependencies between adjacent tokens. This sequential nature makes the inference predominantly memory-bound, as each forward pass requires accessing the complete model parameters as well as Key-Value caches (KV caches). To address the inefficiencies of autoregressive decoding, *Draft-then-Verify* scheme, namely speculative inference, has been introduced [12]–[15]. In speculative inference, multiple future tokens are first *drafted*, followed by parallel *verification* against the target language model (TLM).

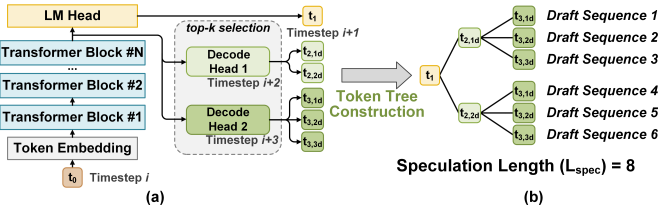


Fig. 2. Tree-based speculative inference and token tree structure.

Speculative inference has been widely studied using various drafting schemes and token structures, as listed in Table I. For drafting schemes, one approach leverages a small DLM to generate draft tokens over several iterations [14], [15], [29], verified in parallel by the original TLM. Alternatively, *self-drafting* [13], [16], [30] augment the TLM itself by integrating additional Feed-Forward Network (FFN) heads (henceforth referred to as Decode Heads) on the final Transformer block, as shown in Fig. 2(a).

For token structures, a conventional sequence-based token structure leads to limited acceptance rate. SpecInfer [12] introduced a tree-based token structure to improve the acceptance length of draft tokens, which selects Top- $k$  draft tokens from each Decode Head and merges them into a token tree by sharing common prefixes. As illustrated in Fig. 2(b), at timestep  $i$ , the original LM head generates one output token for timestep  $i+1$ , while Decode Head 1 and 2 are used to predict future tokens at timestep  $i+2$  and  $i+3$ , respectively. Consequently, Top-2 predictions of Decode Head 1 and Top-3 predictions of Decode Head 2 result in  $2 \times 3$  draft sequences represented as a token tree. The draft tokens are verified in parallel, where a portion of draft tokens are accepted. The parallel verification increases the arithmetic intensity, as the model parameters and KV-caches are accessed only once for multiple draft tokens, which helps to alleviate the memory bandwidth bottleneck in mobile LLM inference.

### III. MOTIVATION

#### A. Analysis of Speculative Inference on PIM Designs

PIM designs leverage high bank-level parallelism to deliver significant throughput advantages for GEMV operations compared to conventional architectures like GPUs. However, speculative inference increases the number of parallel tokens processed during the decoding stage, effectively converting GEMV operations into GEMM operations, degrading the efficiency of prior PIM designs. To quantify this effect, we profile the performance of Samsung LPDDR5-PIM [20] and a mobile NPU for Llama2-7B inference using AttAcc-like data mapping [24]. Detailed configurations are shown in Table II.

Fig. 3 illustrates the performance impact of varying speculation lengths, i.e., the number of speculative tokens, evaluated across 4-die (PIM-4) and 8-die (PIM-8) configurations. For one decoding iteration, PIM-4 achieves  $4.25 \times$  latency improvement and  $15.4 \times$  energy reduction over mobile NPUs, while PIM-8 achieves  $8.34 \times$  latency and  $15.2 \times$  energy gains. However, as the speculation length increases from 1 to 16,

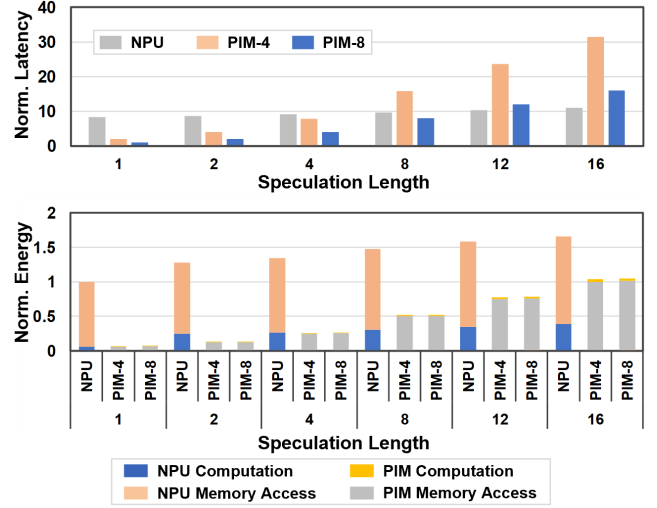


Fig. 3. Speculative inference latency and energy breakdown.

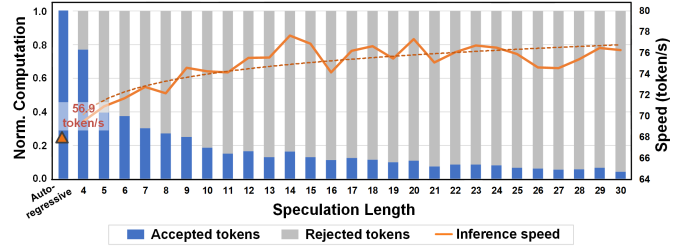


Fig. 4. Profiling of tree-based speculative inference.

both latency and energy advantages deteriorate sharply across PIM configurations.

This performance degradation stems from increased arithmetic intensity of GEMM operations, which improves NPU utilization while eroding bandwidth advantages of PIM designs. Current PIM designs are primarily optimized for GEMV operations, relying on per-bank vector units with limited computational throughput. Their inability to exploit data reuse in GEMM workloads ultimately diminishes effective bandwidth gains for LLM speculative inference.

Although naively increasing the number of PIM devices can also boost overall performance, this approach introduces significant cost and capacity trade-offs. The compute units in PIM devices are fabricated using low-density DRAM process technology, which reduces DRAM memory capacity. For example, Samsung HBM-PIM devices reduce per-die capacity from 8 Gb to 4 Gb [20]. To achieve cost-efficient deployment of speculative inference on mobile devices, it is critical to fully exploit the hardware resources, i.e., both NPU and PIM, through effective workload scheduling. Additionally, architectural optimizations for PIM devices are essential to enhance the performance of GEMM operations.

#### B. Dynamic Token Pruning and Scheduling Opportunities

In tree-based speculative inference, the number of draft tokens grows exponentially with the depth of the token tree. To

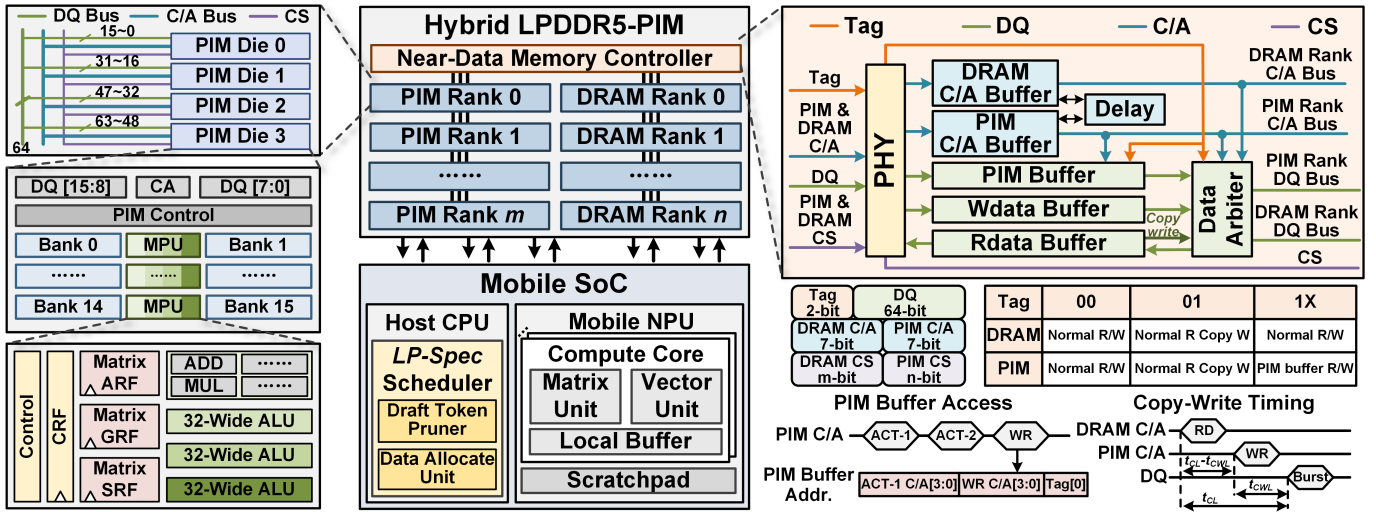


Fig. 5. Overview of *LP-Spec* architecture, the design details of the LP-spec PIM and the near data memory controller.

analyze its inference characteristics, we profile Medusa [16] on an NVIDIA RTX 4090 using randomly generated dense tree structures. It is observed in Fig. 4 that expanding the token tree will increase inference speedup. However, the portion of computation for verifying rejected tokens is also growing. Since rejected tokens are not reused in subsequent decoding iterations, this results in computational and energy waste.

These observations highlight the need for more effective token optimization strategies. Prior approaches have explored techniques such as predetermined optimized token tree structures [16] or using early token pruning mechanisms [13]. However, these methods either fail to capitalize on dynamic optimization opportunities or neglect hardware constraints of PIM-based mobile devices. To address this gap, we explore a hardware-aware draft token pruner which adaptively optimizes the token tree structure to balance inference speed and energy efficiency based on runtime conditions.

Based on the above analysis, we can observe that speculative inference on PIM-based mobile devices remains challenging due to limited computational performance of PIM designs and the redundancies of speculative tokens. In this work, we propose *LP-Spec*, an architecture-dataflow co-optimization approach for efficient speculative inference. At the hardware level, we introduce architectural enhancements including bank-level matrix processing units and a near-data memory controller enabling concurrent PIM and NPU execution. At the software level, we propose dynamic token pruning and scheduling strategies to improve both the performance and hardware utilization of speculative inference on PIM-based mobile platforms.

#### IV. LP-SPEC ARCHITECTURE DESIGN

### A. Architecture Overview

In this work, we present *LP-Spec*, an architecture-dataflow co-design aimed at facilitating efficient speculative inference for LLMs on mobile platforms. As shown in Fig. 5, *LP-Spec*

integrates a mobile SoC with a hybrid LPDDR5-PIM module. The SoC includes both a mobile NPU and a host CPU, with communication between the SoC and PIM facilitated via a 64-bit data bus. The NPU architecture refers to mobile NPUs [21], [22] and features a last-level scratchpad memory and 16 compute cores, each equipped with a SIMD-style matrix unit, a vector unit, and a local buffer. To mitigate the impact of PIM devices on DRAM capacity, the hybrid LPDDR5-PIM module incorporates multiple DRAM and PIM ranks for a total capacity of 16 GB, thus balancing cost and performance. This memory capacity enables the deployment of edge LLMs such as Llama2-7B [31] at INT8 data precision. Each DRAM or PIM rank consists of four dies connected in parallel, operating in lockstep with the same command and address (C/A) and chip select (CS) signals, while utilizing distinct data lines, following JEDEC LPDDR5 specifications [27]. Additionally, the hybrid LPDDR5-PIM module also features a near-data memory controller (NMC), enabling dynamic data reallocation between DRAM and PIM ranks at low cost.

The host incorporates the *LP-Spec* scheduler, which consists of a draft token pruner (DTP) and data allocation unit (DAU). The DTP generates an optimized token tree after each decoding iteration, resulting in varying speculation lengths. The DAU dynamically reallocates model weights between DRAM and PIM ranks through the NMC, ensuring synchronized parallel execution of PIM and NPU devices.

### B. PIM Microarchitecture and Data Mapping

Fig. 5 left details the design of an LPDDR5-PIM die. Building on existing commodity LPDDR5-PIM microarchitecture and instruction set architecture (ISA) [20], [32], we extend and enhance PIM functionality to accelerate speculative inference while addressing the constraints of mobile devices. Each PIM die integrates PIM control logic, 16 banks, and 8 matrix processing units (MPUs), with each MPU shared between two banks. An MPU consists of three key components: (1) four 32-



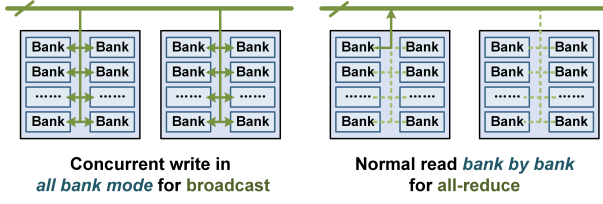


Fig. 6. Comparison of communication overhead between broadcast and all-reduce operations in LPDDR5-PIM architecture.

wide SIMD ALUs, (2) command, general, accumulate, scalar register files (CRF, GRF, ARF, and SRF), and a controller. Given the prevalence of quantized LLM models for edge deployment, the 32-wide ALU features 32 INT8 multipliers and adders. Since an INT8 MAC unit occupies only 26.5% of the area and consumes 63.6% of the energy of a FP16 MAC in 20 nm DRAM process [32], we integrate four 32-wide ALUs while maintaining acceptable hardware cost. The register files consist of 32 32-bit CRFs, 16  $4 \times 256$ -bit matrix GRFs, 16  $4 \times 8$ -bit matrix SRFs, and 8  $4 \times 1024$ -bit matrix ARFs. The ARFs extend precision from INT8 to INT32 for accumulation. The controller fetches PIM instructions from the CRF and schedules the MPU accordingly. Similar to Samsung’s LPDDR5-PIM [20], the MPU supports source operands from both register files and DRAM banks, with bank-sourced data broadcast to all four ALUs for parallel computation. As most power consumption during PIM execution arises from DRAM access ( $> 90\%$ ) [33], the MPU power consumption increases only by 23.2% due to more DRAM data reuse compared with HBM-PIM [29], [32], which remains under the DRAM power budget. When executing PIM computations, all PIM ranks are first switched into all bank mode to load data and instructions, after which they switch into all bank PIM mode to trigger the execution of PIM instructions. This mode switching is achieved by configuring the mode registers.

Considering data mapping in PIM, column-wise partitioning minimizes output data transfer but requires broadcasting of the input data, while row-wise partitioning reduces the input data transfer but involves all-reduce of the output data. Prior work AttAcc [24] employs both row-wise and column-wise partitions for K/V mapping in its customized HBM-PIM, utilizing the accumulators on the bank group and buffer die to perform all-reduce operations. For the LPDDR5-PIM architecture, broadcast operations are easily implemented by enabling all bank mode and activating all CS signals, as shown in Fig. 6. Without on-die accumulators, the PIM execution units can only communicate with other banks on the same die or across different dies through the host. Consequently, all-reduce operations require significantly more data transfer than broadcast operations, resulting in far lower effective bandwidth. For instance, when using 8 PIM dies with 8 compute units per die, all-reduce operations incur  $64 \times$  greater data transfer than broadcast operations. Hence, we adopt column-wise partitioning to minimize communication costs.

### C. Near-Data Memory Controller

Fig. 5 top right shows the architecture design of the NMC. The NMC serves two key functions: enabling concurrent PIM computation and standard DRAM access for the SoC, as well as efficient data reallocation between PIM and DRAM ranks. To support simultaneous PIM computation and DRAM access, the NMC receives independent C/A signals for DRAM ranks and PIM ranks, allowing for parallel operation. Since the SoC can only access either the DRAM memory space or the PIM memory space at any given time, PIM and DRAM can share the same DQ lines. However, larger input matrices in speculative inference may necessitate reloading both the input and the partial sums of the output, leading to contention for the data bus with the NPU. To mitigate this problem, the NMC integrates a dedicated 4 KB PIM global buffer for PIM read/write operations. Unused C/A pins are repurposed as addresses for this global buffer. When execution times between the NPU and PIM become imbalanced, model parameters need to be reallocated between DRAM and PIM ranks. A naive approach would involve reading the model parameters to host and then writing them back to DRAM or PIM ranks, which introduces additional latency and IO energy consumption. In contrast, the NMC introduces a feed-forward path from the read data buffer to the write data arbiter, enabling in-situ copying of data read from DRAM/PIM to PIM/DRAM.

To support NMC functions, additional tag bits are required for guidance, and DRAM timing constraints must be carefully considered. As shown in Fig. 5, we add a 2-bit tag to indicate the operation mode: ‘00’ for normal read/write, ‘01’ for copy-write, and ‘1x’ for PIM global buffer read/write. For PIM global buffer operations, the LSB of the tag combines with an 8-bit address for bank and bank group in ACT-1, WR, and RD commands, forming a 9-bit address to access the PIM global buffer. To ensure proper timing between RD and WR commands in copy-write operations, specific DRAM timing constraints must be adhered to. The interval between issuing an RD command and the start of data transmission is defined as  $t_{CL}$ , while  $t_{CWL}$  specifies the delay between a WR command and the beginning of the write data. Thus, a delay of  $t_{CL} - t_{CWL}$  should be inserted between the RD and WR.

## V. LP-SPEC SCHEDULER

Our observations indicate that substantial energy is wasted on verifying rejected tokens. To enhance runtime efficiency of LLM speculative inference, we propose a hardware-aware draft token pruner (DTP). Based on dynamic speculation lengths, we design a data allocation unit (DAU) to facilitate synchronized NPU-PIM parallel execution. Fig. 7 illustrates the overall framework of the *LP-Spec* scheduler.

### A. Draft Token Pruner

The DTP employs a token tree accuracy model and runtime estimator to dynamically optimize the token tree through iterative sampling and pruning after each decoding step.

**Token Tree Accuracy Model.** We quantify token tree prediction accuracy using expected acceptance lengths for each

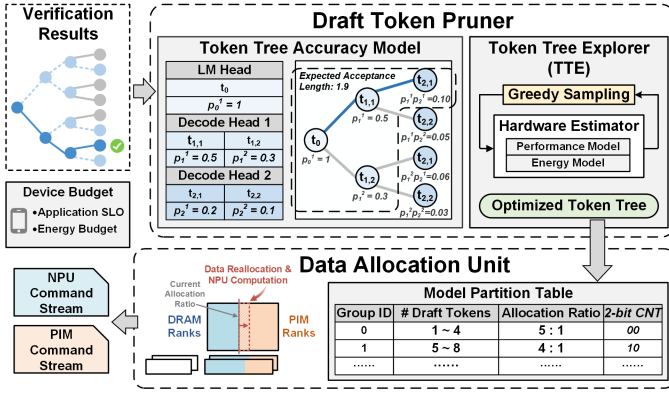


Fig. 7. *LP-Spec* workload scheduler consisting of the draft token pruner (DTP) and the data allocation unit (DAU).

node in tree  $\mathcal{T}$  with  $N$  nodes. As shown in Fig. 7 top, we track the speculation accuracy  $p_i^k$  of the  $k^{th}$  prediction at  $i^{th}$  Decode Head after each decoding step based on previous verification results. For token  $t_i$  in a draft sequence, its expected acceptance length is the product sum of the token accuracy in the sequence, i.e.,  $l_{t_i} = \prod_{i=1}^m p_i^{k_i}$ . For example,  $l_{t_{2,1}} = p_0^1 p_1^1 p_2^1$ . For the entire tree, the expected acceptance length is the sum of all the token nodes. It is worth mentioning that pruning draft tokens does not incur inference accuracy loss.

**Hardware Estimator.** Our hardware estimator combines a performance and energy model to facilitate cost-aware token tree optimization. The performance model quantifies NPU-PIM parallel execution constraints, and total execution latency can be formulated as:

$$T_{total} = \min(T_{NPU}, T_{PIM})$$

The NPU can be modeled using the roofline model, in which its computational throughput is limited by the off-chip memory bandwidth. For PIM devices, the internal bandwidth matches their computation performance. NPU and PIM execution latency can be modeled as follows:

$$T_{NPU} = \frac{N_{params, DRAM}}{BW_{Off-chip}}, T_{PIM} = \frac{N_{params, PIM}}{BW_{PIM}} \times \lceil \frac{L_{spec}}{N_{ALU}} \rceil$$

where  $N_{params, DRAM}$  and  $N_{params, PIM}$  represent weight/KV-cache volumes used in NPU and PIM computation.  $BW_{Off-chip}$  and  $BW_{PIM}$  are the external and internal bandwidth of LPDDR5-PIM,  $N_{ALU}$  is the number of ALUs per MPU, which is 4 in our work.  $L_{spec}$  represents the speculation length. The energy model accounts for PIM and NPU computation, and on-chip/off-chip data transfer.

**Dynamic Token Pruning.** The DTP operates in a closed-loop manner: Verification results from the previous iteration update accuracy statistics of Decode Heads. These statistics, combined with current device resources and optimization objectives, drive the token tree optimization. The token tree explorer (TTE) employs a hardware-informed greedy strategy to construct the optimized token tree from root to leaf. In

each sampling step, the TTE adds the node with the highest prediction accuracy to the tree. Subsequently, the hardware estimator evaluates the performance and energy consumption of speculative inference based on the expected acceptance length of the current token tree. It determines whether to accept the node based on the optimization objectives. This dynamic pruning process yields an optimized token tree for subsequent decoding steps.

## B. Dynamic Workload Scheduling

To maximize the advantages of the heterogeneous system, we adopt NPU-PIM co-processing, in which FC and attention layers are processed by the NPU and PIM using tensor parallelism. Model weights and KV-cache are partitioned between PIM and DRAM ranks. Element-wise and nonlinear functions, such as Softmax and LayerNorm, are executed on the NPU, as implementing these functions in DRAM would incur prohibitive area overhead. During NPU computation, model parameters are fetched from DRAM ranks, while PIM computation utilizes parameters stored in PIM ranks.

To synchronize the execution time of both NPU and PIM devices, the partition ratio of model parameters across PIM and DRAM ranks needs to correlate with the computation throughput of NPU and PIM devices. Since PIM computation throughput depends on  $L_{spec}$ , and the token pruning scheme introduces fluctuations in  $L_{spec}$  across decoding iterations, a fixed model partition ratio can result in sub-optimal performance. To address this issue, we introduce the data allocation unit (DAU), which dynamically adjusts workload mapping and scheduling between the NPU and PIM based on  $L_{spec}$ . As shown in Fig. 7 bottom, DAU computes the optimal allocation ratio for the next decoding iteration based on the model partition table, which stores the optimal allocation ratios for different  $L_{spec}$ . It then generates the corresponding command stream for synchronized NPU and PIM executions.

To strike a balance between excessive parameter reallocation and improved inference latency, we further implement two optimizations in the DAU. *First*, we use a Group ID to select different partition ratios instead of assigning a unique allocation ratio for each  $L_{spec}$ . Multiple  $L_{spec}$  values share the same Group ID, and thus the same allocation ratio. *Second*, the DAU employs a 2-bit saturated counter for each  $L_{spec}$  group to indicate whether the DAU activates. It only activates when  $N_{token}$  exceeds the threshold consecutively twice.

Fig. 8 illustrates two examples of *LP-Spec* dynamic workload scheduling. When the DAU activates, specific model parameters are reallocated between DRAM and PIM ranks. During this reallocation, the NPU reads the weights from the corresponding DRAM/PIM ranks for computation and incurs copy-write operations to migrate the data to PIM/DRAM ranks via the NMC, effectively overlapping the latency between NPU computation and parameter reallocation. Compared to parallel execution without dynamic token-aware workload scheduling, both NPU and PIM can become underutilized as  $L_{spec}$  changes. While the parallel execution during the decoding iteration that involves data reallocation may be imbalanced, the saturated

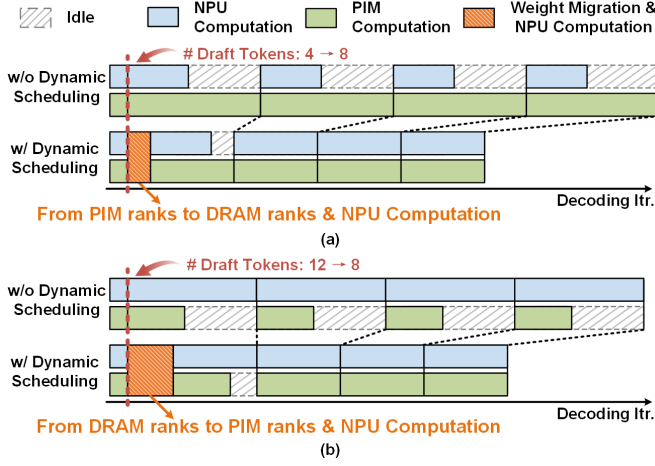


Fig. 8. Static workload scheduling of PIM and NPU and our dynamic workload scheduling in two case: (a) draft tokens increases and (b) draft tokens decreases.

counter ensures that  $L_{spec}$  does not fluctuate dramatically in subsequent iterations, thereby maintaining synchronization between PIM and NPU.

## VI. EVALUATIONS

### A. Experimental Setup

In this work, we implement our *LP-Spec* architecture based on established DRAM vendor designs [23], with specifications listed in Table II. We design *LP-Spec* using  $\times 64$  LPDDR5 to represent typical mobile device scenarios, e.g., laptops and smartphones. Compared with prior PIM designs [23], we enhance the performance by  $4\times$  to 409.6 GOPS for each die. We develop an in-house simulator by integrating and extending the Samsung PIM simulator [34] and LLMCompass [35] for *LP-Spec* system evaluation. Energy consumption data are obtained from prior works [24], [26], [29]. For area evaluation, we use Cadence Genus to synthesize the *LP-Spec* bank-level MPU with a 7nm process [36] at 200 MHz given  $t_{CCD} (5ns)$ . The area overhead for compute units and buffers on the DRAM die was scaled to 1z-nm DRAM process, considering that the DRAM process is  $10\times$  less dense than the logic process [37]. The total area overhead is  $10.31 mm^2$  per DRAM die, which translates to 16.92% of a  $60.92 mm^2$  16 Gb LPDDR5 die [38]. Based on chip microphotograph analysis of DRAM and PIM dies [32], [38], this design reduces memory density by 26.5% compared to Samsung LPDDR5-PIM dies.

In our work, we evaluate our *LP-Spec* against two baselines: (1) Speculative inference on NPU (*NPU-SI*), (2) Speculative inference on GEMV-PIM (*PIM-SI*). Prefill stage of LLM inference and nonlinear functions are executed on the NPU. The baseline mobile NPU is modeled based on commercial products [22] with a total performance of 41 TFLOPS, as detailed in Table II, while Samsung LPDDR5-PIM is used as the GEMV-PIM baseline. For workload evaluations, we adopt the Medusa [16] framework with Llama2-7B and Llama2-13B [39] using INT8 to profile speculative inference on real-world LLM datasets, e.g., Alpaca [40]. Our *LP-Spec* can also be

TABLE II  
HARDWARE SPECIFICATION FOR *LP-Spec*

| NPU Configuration                                                                           |             |                       |                 |
|---------------------------------------------------------------------------------------------|-------------|-----------------------|-----------------|
| Matrix Unit                                                                                 | 32.8 TFLOPS | Vector Unit           | 8.2 TFLOPS      |
| Compute Core / Chip                                                                         | 16          | Local Buffer Capacity | 256 KB          |
| Operation Frequency                                                                         | 1 GHz       | Scratchpad Capacity   | 8 MB            |
| LP-Spec PIM Configuration                                                                   |             |                       |                 |
| # MPU                                                                                       | 8           | # PIM Units / MPU     | 8               |
| MAC Operation Frequency                                                                     | 200 MHz     | Performance           | 409.6 GOPS@INT8 |
| On-chip Bandwidth                                                                           | 51.2 TB/s   | Capacity              | 1 GB            |
| # Die / Rank                                                                                | 4           | Capacity / Rank       | 4 GB            |
| Samsung LPDDR5-PIM Configuration [23]                                                       |             |                       |                 |
| # PIM Unit / Die                                                                            | 8           | Performance           | 102.4 GOPS@INT8 |
| On-chip Bandwidth / Die                                                                     | 51.2 GB/s   | Capacity / Die        | 1 GB            |
| # Die / Rank                                                                                | 4           | Capacity / Rank       | 4 GB            |
| LPDDR5 DRAM Configuration                                                                   |             |                       |                 |
| Off-chip Bandwidth                                                                          | 51.2 GB/s   | Capacity / Die        | 1 GB            |
| # Die / Rank                                                                                | 4           | Capacity / Rank       | 4 GB            |
| PIM / DRAM Timing Parameter                                                                 |             |                       |                 |
| $t_{RP}=15, t_{RCD}=15, t_{RAS}=34, t_{RRD}=4, t_{WR}=28, t_{RC}=30, t_{CCD}=4, t_{FAW}=16$ |             |                       |                 |

implemented using other speculative inference strategies with minimal effort, e.g., DLM-based speculative inference.

### B. Design Benefits of *LP-Spec*

We compare the end-to-end performance of *LP-Spec* against the above baselines with different ( $L_{in}$ ,  $L_{out}$ ) and speculation lengths for two LLMs, in which the DRAM memory configuration is set to 3 PIM ranks and 1 DRAM rank with a total capacity of 16 GB. As shown in Fig. 9(a), *LP-Spec* outperforms the conventional system, achieving on average  $4.59\times$  and  $3.25\times$  over *NPU-SI* and *PIM-SI* baselines, respectively. Compared with the NPU baseline, *LP-Spec* can utilize the high memory bandwidth of PIM while the NPU baseline is limited by the off-chip memory bandwidth. Although *PIM-SI* enjoys the same bandwidth benefits as *LP-Spec*, its computational limitations degrade speculative inference performance, causing its advantage over the NPU baseline to diminish as speculative length increases, which is even worse than *NPU-SI* at the speculative length of 32. However, *LP-Spec* combines high internal memory bandwidth with enhanced compute resources, making it particularly effective for speculative inference across different speculation lengths. Besides, NPU-PIM co-processing and *LP-Spec* scheduling further improve the end-to-end performance of *LP-Spec*. Compared to the naive *LP-Spec*, NPU-PIM co-processing achieves an end-to-end performance improvement up to  $1.47\times$  in average. *LP-Spec* scheduling demonstrates an additional improvement up to  $2.49\times$  in average. When combining them, *LP-Spec* with NPU-PIM co-processing and *LP-Spec* scheduling achieves performance improvement of up to  $13.21\times$  ( $8.47\times$ ) and  $7.91\times$  ( $8.33\times$ ) over *NPU-SI* (*PIM-SI*) baselines for Llama-2 7B and Llama-2 13B. For the following analysis, *LP-Spec* refers to the *LP-Spec* architecture with these two optimizations.

Fig. 9(b) shows the energy consumption of *NPU-SI*, *PIM-SI*, and *LP-Spec*. Compared with *NPU-SI*, *LP-Spec* achieves on average  $7.56\times$  improvement due to less off-chip data transfer. Against *PIM-SI*, *LP-Spec* delivers on up to  $2.85\times$  improvement as our optimized PIM architecture can capture more data reuse opportunities, thereby minimizing the

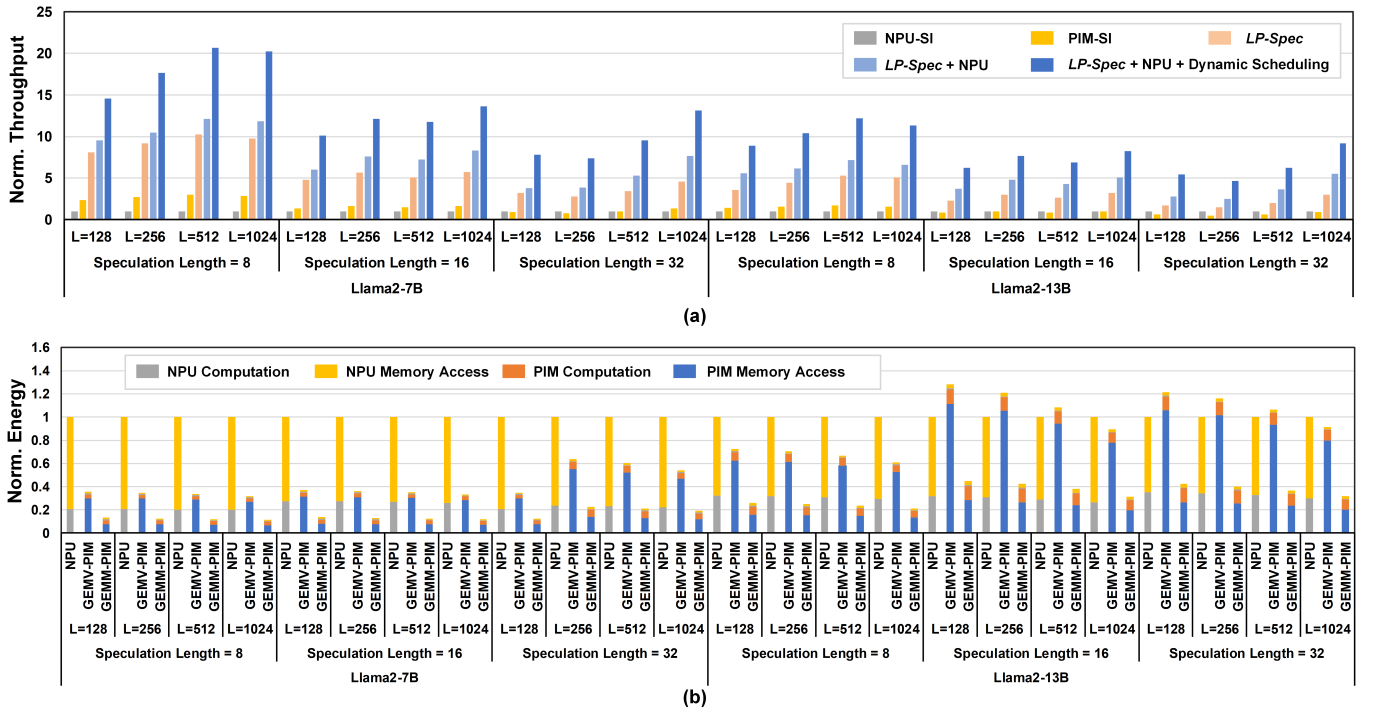


Fig. 9. Throughput and energy efficiency improvements compared with baselines.

TABLE III  
COMPARISON WITH OTHER ACCELERATION HARDWARE

|                                    | AttAcc [24] | RTX 3090  | LP-Spec      |
|------------------------------------|-------------|-----------|--------------|
| <b>Target Scenario</b>             | Cloud       | Mobile    | Mobile       |
| <b>Model</b>                       | GPT-3 175B  | Llama2-7B | Llama2-7B    |
| <b>Throughput (token/s)</b>        | 33.3        | 44.3      | <b>73.4</b>  |
| <b>Energy Efficiency (token/J)</b> | 5.6         | 0.13      | <b>32.6</b>  |
| <b>EDP (s-mJ) ↓</b>                | 5.36        | 173.6     | <b>0.418</b> |

DRAM internal memory accesses. As the speculative length increases, *PIM-SI* advantage over *NPU-SI* degrades. This is because *NPU-SI* can fully reuse weight data to reduce off-chip memory access energy, while *PIM-SI* presents limited reuse opportunities. In contrast, *LP-Spec* replaces vector units in prior PIM designs with MPUs, improving weight reuse rates and reducing internal DRAM memory access overhead. Meanwhile, the DTP in the *LP-Spec* scheduler largely reduces energy on verifying rejected tokens.

### C. Discussions

Prior designs [24], [28], [29] have mainly focused on the PIM-GPU system for cloud services. In these designs, compute-intensive FC layers are offloaded to GPUs, which are equipped with high-bandwidth HBM or GDDR. However, this method cannot be adapted for mobile devices due to low memory bandwidth. Consequently, we develop *LP-Spec* with hardware-dataflow co-optimization architecture to address the above challenges for mobile devices.

As summarized in Table III, *LP-Spec* demonstrates a competitive throughput and a superior energy efficiency of token/J for Llama2-7B inference, which achieves  $12.83\times$  and  $415.31\times$  better EDP than the autoregressive decoding on AttAcc [24] and RTX 3090. Overall, *LP-Spec* is an efficient approach for deploying LLM speculative inference on mobile devices, which can also be extended to other GEMM-dominated workloads, e.g., vision Transformers.

## VII. CONCLUSION

In this paper, we analyzed the challenges of deploying speculative inference on mobile devices, which can not efficiently process GEMM operations on existing hardware. We present *LP-Spec*, a NPU-PIM heterogeneous architecture with GEMM-enhanced LPDDR5-PIM and a dedicated near-data memory controller to accelerate speculative inference and improve the energy efficiency. We also adopt draft token pruning to prune the redundant tokens to improve the energy efficiency and performance. Besides, the dynamic scheduling is proposed to improve hardware utilization of NPU-PIM systems. Evaluations demonstrate that *LP-Spec* achieves up to  $13.21\times$ ,  $7.91\times$  performance and  $7.56\times$ ,  $2.85\times$  energy efficiency gains over the mobile NPU and PIM baselines. Compared with other existing works, *LP-Spec* exhibits  $12.83\times$  and  $415.3\times$  better EDP than AttAcc PIM and 3090 GPU.

## ACKNOWLEDGEMENT

This work was supported in part by NSFC No. 92464202.



## REFERENCES

- [1] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” 2020.
- [2] OpenAI, “Introducing ChatGPT.” <https://openai.com/index/chatgpt/>, 2022.
- [3] GitHub, “Github Copilot.” <https://github.com/features/copilot>.
- [4] Apple, “Apple Intelligence comes to iPhone, iPad, and Mac starting next month.” <https://www.apple.com/newsroom/2024/09/apple-intelligence-comes-to-iphone-ipad-and-mac-starting-next-month/>, 2024.
- [5] Samsung, “Use Live Translate on Galaxy phones and tablets.” <https://www.samsung.com/us/support/answer/ANS10000935/>, 2024.
- [6] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” Nov. 2022. arXiv:2208.07339.
- [7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” May 2023. arXiv:2305.14314.
- [8] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models,” Mar. 2024. arXiv:2211.10438.
- [9] X. Ma, G. Fang, and X. Wang, “LLM-Pruner: On the Structural Pruning of Large Language Models,” Sept. 2023. arXiv:2305.11627.
- [10] E. Frantar and D. Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot,” Mar. 2023. arXiv:2301.00774.
- [11] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A Simple and Effective Pruning Approach for Large Language Models,” May 2024. arXiv:2306.11695.
- [12] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification,” Apr. 2024. arXiv:2305.09781.
- [13] S. Zhong, Z. Yang, M. Li, R. Gong, R. Wang, and R. Huang, “ProPD: Dynamic Token Tree Pruning and Generation for LLM Parallel Decoding,” Feb. 2024. arXiv:2402.13485 version: 1.
- [14] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *40th International Conference on Machine Learning (ICML)*, pp. 19274–19286, July 2023.
- [15] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” Feb. 2023. arXiv:2302.01318.
- [16] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads,” in *International Conference on Machine Learning (ICML)*, 2024.
- [17] H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei, and Z. Sui, “Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2023* (H. Bouamor, J. Pino, and K. Bali, eds.), (Singapore), pp. 3909–3925, Association for Computational Linguistics, Dec. 2023.
- [18] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan, B. Chen, G. Sun, and K. Keutzer, “LLM Inference Unveiled: Survey and Roofline Model Insights,” May 2024. arXiv:2402.16363.
- [19] S. Lee *et al.*, “A 1nm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, pp. 1–3, 2022.
- [20] J. H. Kim, S.-H. Kang, S. Lee, H. Kim, Y. Ro, S. Lee, D. Wang, J. Choi, J. So, Y. Cho, J. Song, J. Cho, K. Sohn, and N. S. Kim, “Aquabolt-XL HBM2-PIM, LPDDR5-PIM with In-Memory Processing, and AXDIMM with Acceleration Buffer,” *IEEE Micro*, vol. 42, no. 3, pp. 20–30, 2022.
- [21] J. Park *et al.*, “A Multi-Mode 8K-MAC HW-Utilization-Aware Neural Processing Unit with a Unified Multi-Precision Datapath in 4nm Flagship Mobile SoC,” in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2022.
- [22] B. Huang *et al.*, “A 5G Mobile Gaming-Centric SoC with High-performance Thermal Management in 4nm FinFET,” in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2023.
- [23] J. H. Kim, Y. Ro, J. So, S. Lee, S.-h. Kang, Y. Cho, H. Kim, B. Kim, K. Kim, S. Park, J.-S. Kim, S. Cha, W.-J. Lee, J. Jung, J.-G. Lee, J. Lee, J. Song, S. Lee, J. Cho, J. Yu, and K. Sohn, “Samsung PIM/PNM for Transformer Based AI : Energy Efficiency on PIM/PNM Cluster,” in *2023 IEEE Hot Chips 35 Symposium (HCS)*, pp. 1–31, Aug. 2023. ISSN: 2573-2048.
- [24] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 103–119, Apr. 2024.
- [25] H. Shin, D. Kim, E. Park, S. Park, Y. Park, and S. Yoo, “McDRAM: Low Latency and Energy-Efficient Matrix Computations in DRAM,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2613–2622, 2018.
- [26] S. Cho, H. Choi, E. Park, H. Shin, and S. Yoo, “McDRAM v2: In-Dynamic Random Access Memory Systolic Array Accelerator to Address the Large Model Problem in Deep Neural Networks on the Edge,” *IEEE Access*, vol. 8, 2020.
- [27] JEDEC, “Low Power Double Data Rate (LPDDR) 5/5X.” <https://www.jedec.org/standards-documents/docs/jesd209-5c>, 2023.
- [28] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, “NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inference,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 722–737, Apr. 2024.
- [29] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, “SpecPIM: Accelerating Speculative Inference on PIM-Enabled System via Architecture-Dataflow Co-Exploration,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 950–965, Apr. 2024.
- [30] M. Stern, N. Shazeer, and J. Uszkoreit, “Blockwise Parallel Decoding for Deep Autoregressive Models,” Nov. 2018. arXiv:1811.03115.
- [31] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” 2023.
- [32] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pp. 43–56, 2021.
- [33] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, “Papi: Exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS ’25, (New York, NY, USA), p. 766–782, Association for Computing Machinery, 2025.
- [34] S. A. I. of Technology, “Pimsimulator.” <https://github.com/SAITPublic>.
- [35] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, “Llmcompass: Enabling efficient hardware design for large language model inference,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 1080–1096, 2024.
- [36] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, “ASAP7: A 7-nm finFET predictive process design kit,” *Microelectronics Journal*, vol. 53, pp. 105–115, July 2016.
- [37] F. Devaux, “The true Processing in Memory accelerator,” in *2019 IEEE Hot Chips 31 Symposium (HCS)*, pp. 1–24, 2019.
- [38] Y.-H. Kim *et al.*, “25.2 A 16Gb Sub-1V 7.14Gb/s/pin LPDDR5 SDRAM Applying a Mosaic Architecture with a Short-Feedback 1-Tap DFE, an FSS Bus with Low-Level Swing and an Adaptively Controlled Body Biasing in a 3rd-Generation 10nm DRAM,” in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 346–348, 2021.
- [39] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” 2023.
- [40] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford Alpaca: An Instruction-following LLaMA model.” [https://github.com/tatsu-lab/stanford\\_alpaca](https://github.com/tatsu-lab/stanford_alpaca), 2023.